

CONSTITUTION

HelixTranslate — Constitution

Status: Active. This document is the project's authoritative rule set. When a rule here conflicts with `CLAUDE.md`, `AGENTS.md`, or any guide, the Constitution wins.

INHERITED FROM `constitution/Constitution.md`

This constitution **extends** the Helix Universal Constitution at `constitution/Constitution.md`. All clauses there apply unless explicitly overridden below with an explicit `Override $X.Y` section. The universal clauses are authoritative for any topic this file does not cover; the project-specific rules below (`CONST-033 ... CONST-036` and the Definition of Done) extend them and never weaken any universal clause. When this file disagrees with the constitution submodule, the constitution wins.

Mission

See `README.md`.

Mandatory Standards

1. **Reproducibility:** every change is reproducible from a clean clone (`git clone <repo> && <project bootstrap>`); no hidden steps.
2. **Tests track behavior, not code:** test what the user-visible behavior is, not what the implementation looks like.
3. **No silent skips, no silent mocks above unit tests.**
4. **Conventional Commits** for all commits.
5. **SSH-only for git operations** (`git@...`); HTTPS prohibited.

Numbered Rules

CONST-033 — Host Power Management is Forbidden

Status: Mandatory. Non-negotiable. Applies to every project, submodule, container entry point, build script, test, challenge, and systemd unit shipped from this repository.

Rule: No code in this repository may invoke a host-level power- state transition (suspend, hibernate, hybrid-sleep, suspend-then- hibernate, poweroff, halt, reboot, kexec) on the host machine. This includes — but is not limited to:

- `systemctl {suspend,hibernate,hybrid-sleep,suspend-then- hibernate,poweroff,halt,reboot,kexec}`
- `logindctl {suspend,hibernate,hybrid-sleep,suspend-then- hibernate,poweroff,halt,reboot}`
- `pm-{suspend,hibernate,suspend-hybrid}`
- `shutdown {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}`
- DBus calls to `org.freedesktop.login1.Manager.`
`{Suspend,Hibernate,HybridSleep,SuspendThenHibernate,PowerOff,Re boot}`
- DBus calls to `org.freedesktop.UPower.`
`{Suspend,Hibernate,HybridSleep}`
- `gsettings set ... sleep-inactive-{ac,battery}-type` to any value other than 'nothing' or 'blank'

Why: The host runs mission-critical parallel CLI-agent and container workloads. On 2026-04-26 18:23:43 the host was auto- suspended by the GDM greeter's idle policy mid-session, killing HelixAgent and 41 dependent services. Recurring memory- pressure SIGKILLS of `user@1000.service` (perceived as "logged out") have the same outcome. Auto-suspend, hibernate, and any power-state transition are unsafe for this host.

Defence in depth (mandatory artifacts in every project):

1. `scripts/host-power-management/install-host-suspend-guard.sh` — privileged installer, manual prereq, run once per host with `sudo`. Masks `sleep.target`, `suspend.target`, `hibernate.target`, `hybrid-sleep.target`; writes `AllowSuspend=no` drop-in; sets `logind IdleAction=ignore` and `HandleLidSwitch=ignore`.
2. `scripts/host-power-management/ user_session_no_suspend_bootstrap.sh` — per-user, no-sudo defensive layer. Idempotent. Safe to source from `start.sh` / `setup.sh` / `bootstrap.sh`.
3. `scripts/host-power-management/check-no-suspend-calls.sh` — static scanner. Exits non-zero on any forbidden invocation.
4. `pkg/challenge_runner/scripts/host_no_auto_suspend_challenge.sh` — asserts the running host's state matches layer-1 masking.
5. `pkg/challenge_runner/scripts/no_suspend_calls_challenge.sh` — wraps the scanner as a challenge that runs in CI / `run_all_challenges.sh`.

Enforcement: Every project's `CI / run_all_challenges.sh` equivalent MUST run both challenges (host state + source tree). A violation in either channel blocks merge. Adding files to the scanner's `EXCLUDE_PATHS` requires an explicit justification comment identifying the non-host context.

See also: docs/HOST_POWER_MANAGEMENT.md for full background and runbook.

CONST-034 — LLMsVerifier is the Single Source of Truth for All LLM Models

Status: Mandatory. Non-negotiable. Applies to all translation workflows, API endpoints, CLI commands, distributed workers, and WebSocket sessions.

Rule: LLMsVerifier SHALL be the EXCLUSIVE and ONLY source of truth for all LLM models available to HelixTranslate. No model may be used in HelixTranslate that does not originate from LLMsVerifier and pass its verification gate.

Requirements:

1. **F-SSOT-001:** Only models verified by LLMsVerifier are eligible for use.
2. **F-SSOT-002:** HelixTranslate MUST NOT maintain any independent model registry, hardcoded model lists, or fallback model definitions outside of LLMsVerifier-provided data.
3. **F-SSOT-003:** All model metadata (name, ID, capabilities, token limits, pricing, features) SHALL be retrieved from LLMsVerifier at runtime or build time.
4. **F-SSOT-004:** Model information SHALL be cached locally with configurable TTL, but cache invalidation MUST trigger re-fetch from LLMsVerifier.
5. **F-SSOT-005:** If LLMsVerifier is unreachable, HelixTranslate SHALL either use expired cache with warning or enter degraded mode with explicit user notification. No unauthorized models SHALL be used.
6. **F-GATE-001:** ONLY models that have PASSED LLMsVerifier validation, verification, and scoring pipeline SHALL be presented to end users.
7. **F-GATE-002:** Models MUST satisfy ALL three criteria: (a) validation passed, (b) verification passed, (c) positive scoring (> 0 overall score).
8. **F-GATE-003:** The scoring algorithm: $\text{Overall Score} = (\text{Responsiveness} \times 0.30) + (\text{Code Capability} \times 0.25) + (\text{Feature Richness} \times 0.25) + (\text{Reliability} \times 0.20)$. Models with score ≤ 0 SHALL be rejected.
9. **F-GATE-004:** Models MUST have `VerificationStatus == "verified"` AND `CanSeeCode == true` to be eligible.
10. **F-GATE-005:** Models MUST have `AffirmativeResponse == true` to be eligible.

Enforcement: Any code change that introduces a hardcoded model list, bypasses the verification gate, or uses an unverified model constitutes a Constitution violation and MUST be reverted.

CONST-035 — Anti-Bluff Testing Constitution (MANDATORY)

Status: Mandatory. Non-negotiable. Applies to every test, challenge, HelixQA bank entry, and CI pipeline in this repository and all submodules.

§35.1 The Problem (Historical Mandate)

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used!"

This is the **worst possible outcome**: green tests + broken features. Every rule below exists to prevent this.

§35.2 The Six Anti-Bluff Rules

Every test, Challenge, and HelixQA bank entry MUST:

1. **Assert on a concrete end-user-visible outcome** — translated text content, DB row, downloadable file, visible dashboard element, API response body. NOT just "no error" or "200 OK" or "exit code 0".
2. **Run against the real system** — mocks ONLY in unit tests (`go test -short`). ALL other test types MUST use real services, real databases, real LLM providers (or documented SKIP-OK: `#<ticket>`).
3. **Include a matching negative assertion** — test MUST fail when the feature is broken. E.g., if testing "English→Serbian translation", also assert the output is NOT in English.
4. **Emit copy-pasteable evidence** — response body snippet, screenshot filename, DB row dump, log excerpt, video timestamp.
5. **Verify "fails when feature is removed"** — deliberately break the feature (comment out implementation, change API key to invalid), re-run test, test MUST FAIL.
6. **No blind shells** — no `&& echo PASS`, no `|| true`, no `tee` exit-code laundering, no `test -f file && echo "PASS"` without checking file content.

§35.3 HelixQA-Specific Anti-Bluff Rules

- Bank entries declare **executable actions** (never prose).
- Each entry declares **concrete success predicates**: `assertBodyContains: 'translated text', assertVisible: 'Translation Complete'`.
- **Stagnation guard** — frame N+1 identical to N for >10 seconds = FAIL.
- Vision-model `verified=true` with empty/tautological reasoning = INCONCLUSIVE (not PASS).
- `IsBlankScreenshot()` must pass before any vision analysis.

§35.4 Functional Probe Floor

- TCP-open is the FLOOR, not the ceiling.
- PostgreSQL → `SELECT 1` returns 1.
- Redis → `PING` returns PONG.
- API → `GET /health` returns `{"status": "healthy"}` with non-empty body.
- Translation → actual translated text in response, not just `session_id`.

§35.5 Evidence Requirements

- Every PASS must carry positive evidence captured during execution.
- No metadata-only PASS, no configuration-only PASS, no "absence-of-error" PASS.
- Evidence types: API response bodies, downloaded files, database query results, browser console output, screenshots, video frames.

§35.6 Bluff Taxonomy (FORBIDDEN Patterns)

Bluff Type	Example	Why It's Wrong
Wrapper bluff	Test asserts function returns, but caller ignores return value	Passes but feature unused
Contract bluff	System advertises capability but rejects it in dispatch	Advertises what it can't do
Structural bluff	Checks file exists but doesn't verify content	File exists but is empty/corrupt
Comment bluff	Code comment promises behavior code doesn't have	Tests comment, not code
Skip bluff	<code>t.Skip("not running yet")</code> without SKIP-OK marker	Hides broken test

§35.7 Mutation Testing (Mandatory)

- Every challenge **MUST** have a paired mutation test.
- Mutation deliberately breaks the feature → the challenge **MUST** then FAIL.
- A challenge without a paired mutation = BLUFF challenge = Constitution violation.

§35.8 Audit Ritual

Every Full-QA cycle **MUST**:

1. Pick 5 random tests + 5 random challenges.
2. Comment out the target implementation.
3. Re-run tests/challenges.
4. Confirm they FAIL.
5. Restore implementation.
6. Document results in session report.

§35.9 User Mandate (NON-NEGOTIABLE)

The bar is NOT "tests pass" but **"users can use the feature."** A translation that "completes" but produces garbage text is a FAIL. An API that returns 200 but with wrong content is a FAIL. A dashboard that "loads" but shows no data is a FAIL.

CONST-036 — HelixQA is the Sole Authorized QA Tool

Status: Mandatory. Non-negotiable.

Rule: All automated UI/UX testing of the Web Dashboard and API endpoints **MUST** use HelixQA. No custom Playwright scripts, no curl-based test harnesses outside HelixQA banks.

Requirements:

1. **HelixQA-only for Web Dashboard and API testing.**
2. **Vision-driven only.** Screenshot → LLM analysis → action decision. No hardcoded selectors, no sleep timers.
3. **Universal Solution Principle.** When HelixQA cannot interact with a HelixTranslate UI element, the fix **MUST** be implemented in HelixQA itself, never by adding test hooks to HelixTranslate.

4. **Live log monitoring.** Every session streams API logs, gRPC logs, translation logs.
5. **Screen-state tracking.** Frame N vs N+1. Stagnation >10s = critical failure.
6. **Executable actions in banks,** never prose.
7. **Video mandatory for Web Dashboard sessions.** Screenshots at every step.
8. **Evidence validation.** Post-translation must contain actual translated text, not placeholder.

Definition of Done

A change is done when:

1. The code change is committed.
2. All project-level tests pass on a clean clone.
3. All challenges in pkg/challenge_runner/scripts/ pass on the running host.
4. Governance docs (CONSTITUTION.md, AGENTS.md, CLAUDE.md) are coherent with the change.
5. All HelixQA banks pass for all configured platforms.
6. Anti-bluff audit (CONST-035 §35.8) passes — random tests confirmed to fail when feature removed.
7. Every modified/added feature has a registered challenge.
8. Evidence from the most recent QA session is archived in docs/reports/qa-sessions/.

See also

- README.md — project overview, quickstart.
- AGENTS.md — guidance for AI coding agents (Codex, Cursor, etc.).
- CLAUDE.md — guidance specifically for Claude Code.
- docs/HOST_POWER_MANAGEMENT.md — CONST-033 background and runbook.
- docs/research/llms_verifer/ — LLMsVerifier integration materials.
- docs/research/helix_qa/ — HelixQA integration materials.

CONST-035 — Anti-Bluff Operative Rule (MANDATORY)

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This **MUST NOT** be the case."

The operative rule: Execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product.

- A green test or challenge for a feature that does not actually work is a **BLUFF** and is **FORBIDDEN**.
- Every test must assert concrete user-visible outcomes, not just internal state.
- Every challenge must run real code and verify real behavior; grep/file-existence checks are **NOT sufficient**.
- Mutation testing is **MANDATORY**: deliberately break the feature → the test/challenge **MUST then FAIL**.

- The bar for shipping is **NOT "tests pass"** but **"users can use the feature."**
- No false-success results are tolerable.